CVE-Guided Typestate Analysis for Recurring Vulnerability Detection

RUTVIJ WAGH, Indian Institute of Technology Guwahati, India
UDAYRAJ CHOUDHARY, Indian Institute of Technology Guwahati, India

A public vulnerability fix teaches a lifecycle rule: an object must not be released without resetting a caller-visible reference to it, or without detaching it from a chain it still belongs to. Existing general-purpose static analyzers do not check a specific project's own such conventions; they check their own stock rule sets instead. CVE-2022-4450 is a real case: `PEM_read_bio_ex` in OpenSSL frees two output buffers on one failure path without clearing the caller-visible pointers to them, letting a caller double-free. We present Tscк, which reads such a rule from a data file and checks it against a `tree-sitter` control-flow graph. It flags every release site with no path-complete reset, each with a witness path a human can check by hand. On OpenSSL, Tscк flags `PEM_read_bio_ex`'s two outputs before the fix and clears both after it. It also reproduces an unrelated non-CVE fix the rule was never shown, and flags one site a later OpenSSL commit went on to fix independently. On the current 1,155-file `crypto/ssl/providers` tree it flags 10 sites: manual triage finds 2 potential recurrences whose failure branch is not reachable in normal builds, 8 harmless deviations, and 0 confirmed new vulnerabilities. None of three general-purpose open-source static analyzers we ran, with their stock configuration, distinguishes any of three historical CVEs from their own fix; a project-specific rule, not a sharper general one, closed that gap. A second, differently-shaped rule for the detach-before-release BIO-chain invariant behind CVE-2023-0215 reuses the same parser and control-flow graph and also distinguishes its own motivating fix. Its whole-tree run, though, flags 11 sites that are all false positives, because the rule has no notion of which released object is externally owned. That is a concrete limit on how far a single-object rule generalizes, even under the same architecture that generalized cleanly for the first rule.

CCS Concepts: • **Software and its engineering** → **Automated static analysis**; • **Security and privacy** → *Software and application security*.

Additional Key Words and Phrases: static analysis, typestate analysis, control-flow graph, recurring vulnerabilities, OpenSSL, memory safety, output parameters, tree-sitter

ACM Reference Format:
Rutvij Wagh and Udayraj Choudhary. 2026. CVE-Guided Typestate Analysis for Recurring Vulnerability Detection. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (Format Template) (FSE (CS-5187 Term Project))*. ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Suppose you have just read a public vulnerability fix for a C library you depend on: a one-line patch that frees a buffer and then clears the pointer to it, because the code path that frees it can be reached again before the function returns. You trust the fix. The natural next question is whether the same mistake, the same missing reset after the same kind of release, sits somewhere else in the same codebase, in a function the patch never touched.

Static analyzers already look for memory-safety bugs in C, and some are free and widely used: Clang Static Analyzer, Cppcheck, and Coccinelle semantic patches, among others. These tools ship with general rule sets built from common bug patterns across many projects. They stop short of one thing: a rule derived from a single project's own patch history, for the exact ownership or reset

Authors' Contact Information: Rutvij Wagh, Indian Institute of Technology Guwahati, India; Udayraj Choudhary, Indian Institute of Technology Guwahati, India.

FSE (CS-5187 Term Project), IIT Guwahati
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

convention that patch teaches, checked everywhere that convention should hold in that project's current source.

We focus on a second problem that starts where general-purpose rule sets stop. Given one historical fix, can a rule extracted from it, with no function names baked in, be checked against a project's whole source tree? We ask three things of such a rule: (i) flag the function the fix came from before the fix, and clear it after; (ii) do the same for an unrelated fix nobody built the rule from; and (iii) say, for every site it flags on the project's current source, whether that site is conforming, a real violation, or explainable, with a witness a human can check by hand.

CVE-2022-4450 is a real case. `PEM_read_bio_ex` in OpenSSL's `crypto/pem/pem_lib.c` frees two output buffers on one failure path (Figure 1) without clearing the caller-visible pointers to them; a caller that frees these outputs again after failure can double-free. The fix, commit `63bcf189be`, adds exactly two assignments. This is not a single case: applying the same rule, unchanged, to OpenSSL's current source tree (1,155 C files across `crypto/`, `ssl/`, and `providers/`) flags 10 release sites with the same shape (Section 5).

Closest to our rule-as-data approach is VulPA [Cao et al. 2025a], which also derives a checkable rule from a known bug, but compiles it to an automaton checked by an interprocedural analysis with aliasing, on Java. Our rule is intraprocedural, has no alias analysis, and is checked on one rule family; Section 8 states this gap, and three others, in full.

We present Tsck, a checker that reads a lifecycle rule from a data file and applies it to real C source through a parser and a per-function control-flow graph, rather than matching one patch's text.

(1) It builds a control-flow graph for every function with `tree-sitter` (Section 4.2).
(2) For every output parameter a function releases, it searches the graph for a path to function exit that never resets it (Section 4.3).
(3) It classifies each release site as conforming, a violation, or suppressed, with a witness path for every flagged site (Section 4.4).

This paper implements two such rules against the same parser and CFG builder: Protocol B, for the output-pointer reset convention behind CVE-2022-4450, and Protocol A, for the detach-before-release BIO-chain convention behind CVE-2023-0215 (**RQ5**, Section 5.9).

On OpenSSL, Tsck flags `PEM_read_bio_ex`'s two output parameters before the fix and clears both after it. It also reproduces an unrelated non-CVE fix the rule was never shown, and flags one site a later OpenSSL commit went on to fix independently (Section 5). On the current source tree it flags 10 sites. Manual triage finds 2 potential recurrences whose failure branch is not reachable in normal builds, 8 harmless deviations, and 0 confirmed new vulnerabilities. The lesson we did not expect going in: none of three general-purpose open-source static analyzers, run with their stock configuration, distinguishes any of the three CVEs we checked from its own fix (Section 5.6). A project-specific rule, not a sharper general one, is what closed the gap here. Protocol A also distinguishes its own motivating fix, but its whole-tree run finds 0 confirmed recurrences out of 11 flagged sites, all false positives from the rule's missing notion of object ownership: the rule-as-data architecture generalizes across invariant shapes even where a given rule's detection does not generalize across a whole codebase (Section 5.9).

Contributions.

- We define a rule-as-data lifecycle invariant for output-pointer reset after release, independent of any function name (Section 3).
- We present Tsck, which checks this rule against a real `tree-sitter` control-flow graph instead of patch text (Section 4).

```
headerlen = BIO_get_mem_data(headerB, NULL);
*header = pem_malloc(headerlen + 1, flags);
*data = pem_malloc(len, flags);
if (*header == NULL || *data == NULL)
    goto out_free;
...
out_free: // *header, *data freed, never reset
    pem_free(*header, flags, 0);
    pem_free(*data, flags, 0);
end:
    ...
    return ret; // caller sees stale *header, *data
```

Fig. 1. `PEM_read_bio_ex` in OpenSSL 8e257b86e5 (`crypto/pem/pem_lib.c`, trimmed). The `out_free` path frees `*header` and `*data` but never resets either pointer before `return`. The fix, commit `63bcf189be`, adds `*header = NULL;` and `*data = NULL;` to this path; the same rule, applied unchanged to OpenSSL's current 1,155-file `crypto/ssl/providers` tree, flags 10 sites with this shape (Section 5).

- We evaluate Tsck on six pinned OpenSSL snapshots and compare four tools, including Tsck's own Part 1 baseline, against three historical CVEs (Section 5).
- We implement a second, differently-shaped rule, Protocol A, for the BIO-chain invariant behind CVE-2023-0215, and show where a single-object rule's generalization breaks down even under the same rule-as-data architecture (Section 5.9).
- We release the tool and all data (Section 9).

2 Background

Output parameter. A C function can return a value through a parameter of pointer-to-pointer type, written `T **P`. The caller passes the address of its own pointer variable; the function writes a value into `*P` for the caller to read after the call returns. We write `*P` for the object the caller's pointer refers to, not for `P` itself.

Release and reset. A *release* of `*P` is a call that frees the object `*P` points to, for example `OPENSSL_free(*P)`. After a release, `*P` still holds the freed address unless something changes it. A *reset* of `*P` is an assignment that gives it a new value, most often `*P = NULL`. A dangling `*P`, one that still holds a freed address after a release and before any reset, is dangerous exactly when the caller can read or free it again: a double free, or a use of freed memory if the caller dereferences it.

Typestate. *Typestate analysis* tracks not just an object's type but which operations are currently valid on it, as a small state machine. [TODO: Cite the originating typestate paper once its author list and venue are verified directly (for example on IEEE Xplore); automated search returned conflicting author lists for this title across mirrors and we do not cite it unverified.] For the invariant in this paper, `*P` moves through three states: *Valid* before any release, *Released* after a release and before a reset, and back to *Valid* (we call it *Reset*, to keep the release that caused it visible in our output) once a reset occurs. A *violation* is a program path on which `*P` can reach function exit while still in the *Released* state. CVE-2022-4450 (Section 5.5) is exactly this: `PEM_read_bio_ex` frees `*header` and `*data` on one failure path and returns without resetting either.

Release site. We call the specific call expression that performs a release, together with its source location and the parameter `P` it releases, a *release site*. Section 5 reports counts of release sites, not of functions: one function can contain several release sites, one per output parameter it releases.

Rule as data. Existing patch-matching checkers, including the Part 1 baseline in this project (`typestate_check.py`), encode a rule as a sequence of string or syntax checks written for one named function. We instead write the rule once as data (Section 4.1), independent of any function name, and apply it to every function in a source tree that has a parameter of the tracked shape.

3 Problem Statement

We now state the problem precisely. Let f be a C function with a parameter P of pointer-to-pointer type or deeper. Let r be a *release site* inside f: a call that frees the object $*P$ points to (Section 2). A *reset* of $*P$ is an assignment to $*P$, or, when the rule allows it, passing P itself to a call that may reassign it. Let paths(r) be the set of control-flow paths from r to the exit of f, built from f's real parse tree (Section 4.2).

Goal. r is *conforming* if every path in paths(r) passes a reset of $*P$ before reaching exit. r is a *violation* if some path in paths(r) reaches exit without passing a reset. Given a rule (Section 4.1) and a C source tree, our goal is to compute this conforming/violation label, with a witness path, for every release site whose parameter matches the rule.

This is hard for two reasons that a string-matching checker, such as the Part 1 baseline in this project, sidesteps by only looking at one named function. First, paths(r) can branch and loop arbitrarily, so checking every path by hand does not scale past a handful of functions. Second, a reset on one branch and none on a sibling branch is indistinguishable from a reset in comments or dead code to a tool that only searches for a matching substring; only a parse tree and a control-flow graph can tell them apart.

Why not just grep for the release/reset pair? A regular expression can check whether a function's text contains both a release call and a reset assignment, which is what `typestate_check.py` does for three named functions. It cannot tell whether the reset lies on every path from the release to exit, or only on one branch that a bug report's own failure path never takes; Section 5 needs that distinction to call a site conforming or a violation, so we build the control-flow graph instead of matching text.

4 Approach

Figure 2 shows the steps of Tsck. The main idea is to read a lifecycle rule from a data file, rather than hard-coding it. Tsck checks that rule against a per-function control-flow graph built from the real parse tree of each C source file. This paper implements two such rules against the same parser and CFG builder: Protocol B (Section 4.3), a release/reset rule derived from CVE-2022-4450, and Protocol A (Section 5.9), a differently-shaped establish/detach-before-release rule for the BIO-chain invariant behind CVE-2023-0215. We return to what each rule does and does not generalize to in Section 7.

4.1 The rule, as data

The rule for Protocol B is a JSON file, not a code path: it names no OpenSSL function. It tracks parameters of pointer-to-pointer type; a *release* is any call whose name matches the regular expression `(?i)free` and whose argument is `*P`; a *reset* is an assignment to `*P`, or, by a configurable flag, passing `P` itself to a later call. The invariant is: if a function releases `*P`, every path from that release to function exit must reach a reset first. We do not claim the idea of an output-parameter reset invariant as new; what we built is a parser and path-search engine that checks it against real source, rather than against one patch's text.

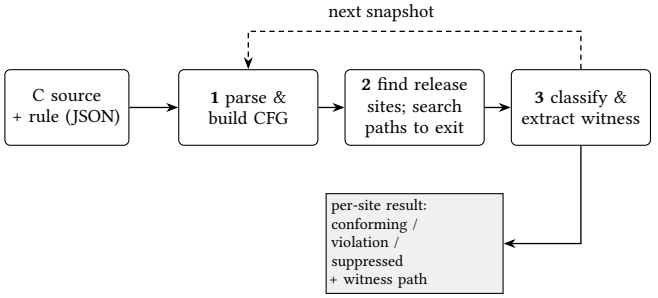


Fig. 2. Tsck takes a C source tree and a rule stored as data (Section 4.1). It parses every file once, finds every site that releases a tracked pointer, and searches forward in the control-flow graph for a path to function exit that never resets it. Each flagged site keeps a witness path. The outer loop (dashed) reruns this per pinned OpenSSL snapshot; nothing loops within one file's analysis.

4.2 Step 1: Parsing and the control-flow graph

Tsck parses each C file with `tree-sitter` and `tree-sitter-c`. We do not claim parsing itself as new; everything from here on is. For each function, we build a control-flow graph whose nodes are simple statements, branch and loop conditions, labels, and join points; `goto` targets, including error labels such as `err:`, become real edges, so an error path that jumps past a reset is visible to the path search in Step 2. Three approximations keep the graph tractable and are stated as threats to validity (Section 7): a compound condition joined by `&&` or `||` is one node, without separate short-circuit edges; a `#if`/`#ifdef` block is treated as a branch, keeping both variants rather than resolving the macro; and macro bodies are not expanded, so a macro call is treated as an ordinary call expression.

4.3 Step 2: Release sites and the path search

For every parameter `P` of pointer-to-pointer type or deeper, Tsck scans each control-flow node for calls matching the release pattern and for assignments matching the reset pattern. From every release site, it searches the control-flow graph forward, breadth-first, for a path to function exit. The search stops following a branch once it crosses a reset, and the release's own dereference of `*P` licenses one pruning rule: because the release itself has already dereferenced `*P`, `P` is non-null on every path after it, so a later branch that is live only when `P == NULL` is dead code and is not explored. If the search reaches function exit without crossing a reset, the site is a candidate violation and the path taken becomes its witness.

Two refinements narrow this candidate set. We disclose both as chosen after reading real output, not derived from the rule alone. *Deref- after-release suppression* drops a site where `*P` is read again before any reset. This signals a partial release and reuse, not a dangling pointer: for example, a function that frees only part of a struct's contents and then reinitializes it. The second refinement is the `NULL`-guard pruning described above, added after it produced one infeasible-path false positive on a whole-tree scan. Both are unit-tested (Section 5) and are listed, not hidden, in each result.

4.4 Step 3: Classification and witness extraction

Each release site is labeled *conforming* if every path from it to function exit passes a reset; *violation* if some path reaches exit without one; or *suppressed* if the path rule would flag it but one of the two refinements in Step 2 applies. A suppressed site is reported, with its reason, alongside conforming and violation sites; it is never dropped silently. For a violation, Tsck records the source lines on the witness path. A reader can then check the finding against the source without re-running the tool.

`compare.py` diffs two runs of the same files at different revisions and reports exactly which sites a given commit removed or added. Section 5 uses this to measure whether a real fix changes only the sites its own diff touches.

5 Evaluation

We ask five questions.

RQ1 Does Tsck (the invariant-based Protocol B engine) detect the motivating CVE on real OpenSSL source, distinguish the fix from the bug, and generalize to a fix it was not derived from?

RQ2 Do general-purpose open-source static-analysis tools already catch the bugs a source-pattern baseline catches, on the same three CVEs?

RQ3 Does a rule written from one CVE flag a site that OpenSSL fixed only later, without the rule being changed after the fact?

RQ4 What does Tsck cost in time?

RQ5 Does Tsck's rule-as-data design extend to a second, differently-shaped invariant (Protocol A, CVE-2023-0215), and does that second rule generalize the same way Protocol B's did?

5.1 Subjects

We select three historical OpenSSL CVEs and one non-CVE fix, each as a pinned vulnerable/fixed commit pair, with the following criteria:

(1) a memory-safety fix with a clear, publicly documented root cause (official advisory or commit message), so the ground-truth label is not our own judgment;
(2) source available at both the vulnerable and the fixed revision in the same file, so a rule can be checked pre- and post-fix without reconstructing context;
(3) at least one case not used to derive the rule (**RQ3**), so the evaluation is not entirely circular.

CVE-2022-4450 (`PEM_read_bio_ex`, fix `63bcf189be`) is the motivating example and the source of the Protocol B rule. CVE-2023-0215 (`BIO_new_NDEF`, fix `8818064ce3`) is both the baseline comparison's second CVE (**RQ2**) and the source of Protocol A (**RQ5**), a second rule outside Protocol B's release/reset shape. CVE-2024-4741 (`SSL_free_buffers`, fix `b3f0eb0a29`) is included in the baseline comparison only: it needs a state guard that neither rule's shape can express, and we report that scope honestly rather than silently dropping the case where Tsck does not apply. The `d2217c88df` KDF fix (not a CVE, a different author, a year before CVE-2022-4450) is the out-of-sample case for **RQ3**.

5.2 Baselines

Part 1 pattern baseline. `typestate_check.py`, the project's own string/regex patch matcher from the first part of this project. It is included, not as a baseline for Tsck, but because it is the simplest thing that could work: if general-purpose tools cannot even match a patch-specific string check, that is informative on its own.

Clang Static Analyzer 18. A general-purpose open-source C/C++ analyzer, run with `scan-build-18` on the affected translation unit of each pinned revision, with the revision's own generated OpenSSL build flags.

Coccinelle 1.1.1 (spatch). Five pre-existing semantic patches (*not* written for these three CVEs) for release/use, repeated release, release-after-transfer, and allocate-then-return patterns.

Cppcheck 2.13.0. Run with `--enable=warning,style,performance,portability --inconclusive` on each affected file, C11/unix64, with the revision's generated headers.

Table 1. Tscк (Protocol B) on six pinned OpenSSL snapshots (`crypto/`, `ssl/`, `providers/`). A *release site* is a call that frees `*P` for a pointer-to-pointer parameter `P`. The two fixes each remove exactly their own violations and add none.

Snapshot	C files	Conforming	Violation	Suppressed
CVE-2022-4450 pre-fix (`8e257b86e5`)	1001	99	7	2
CVE-2022-4450 fix (`63bcf189be`)	1001	101	5	2
KDF fix parent (`29a27cb2c5`)	1005	93	12	2
KDF fix (`d2217c88df`, not a CVE)	1005	97	8	2
OpenSSL 3.0.0 (`89cd17a031`)	999	92	10	2
master (`949fe8b5f4`, 2026-10-30)	1155	132	10	2

None of the three general-purpose tools was configured or tuned toward these three CVEs; their rule sets and query sets are their stock general-purpose offering. This comparison measures detection by a general-purpose tool against a baseline source check, not against Tscк itself, because Tscк (Protocol B) does not yet implement a rule for two of the three CVEs (Section 5.1).

5.3 Measurements

RQ1. Release-site counts (conforming / violation / suppressed) from Tscк on six pinned OpenSSL snapshots spanning `crypto/`, `ssl/`, `providers/`, and whether each of two real fixes removes exactly its own violations and adds none.

RQ2. For each of the four tools and each of the six revisions (three CVEs × two sides), whether the tool's output distinguishes the vulnerable revision from the fixed revision. We read this manually against each fix commit.

RQ3. Whether Tscк, run unchanged after being derived from CVE-2022-4450, flags a site in an unrelated, later-dated fix tree that OpenSSL went on to fix one commit afterward.

5.4 Setup

Tscк (Protocol B) uses Python 3.13, `tree-sitter` 0.26.0, and `tree-sitter-c` 0.24.2 on Linux (kernel 6.6.87.2, WSL2), git 2.43.0. The tool-comparison baselines (**RQ2**) ran on a separate machine (Linux, WSL2) with Python 3.12.3, Clang/`scan-build` 18.1.3, Coccinelle (`spatch`) 1.1.1 on OCaml 4.14.1, and Cppcheck 2.13.0; exact executable SHA-256 hashes and frozen rule files are recorded alongside this evaluation's data. We do not mix numbers from the two environments in one measurement; **RQ1** and **RQ3** use only the first, **RQ2** only the second. For **RQ4**, a second author independently reran `run_scan.sh` on a third machine (Python 3.11.6, same `tree-sitter`/`tree-sitter-c` versions, kernel 6.18.54.1-microsoft-standard-WSL2) under `/usr/bin/time -v`; the timing log is recorded alongside this evaluation's data. For **RQ5**, Protocol A's motivating-pair and whole-tree runs were each independently reproduced on a fourth environment (`tree-sitter` 0.26.1, `tree-sitter-c` 0.24.2), reproducing identical counts to the first run; see Section 9 for who ran which environment.

5.5 RQ1: Does Tscк detect and generalize the motivating CVE?

Table 1 shows Tscк on the motivating function and on the whole tree. At the vulnerable revision, Tscк flags `PEM_read_bio_ex`'s `*header` and `*data` as violations (witness lines 991–998); at the fix, both sites are conforming. The same fix removes exactly these two violations from the whole-tree count (99 → 101 conforming, 7 → 5 violation) and adds none.

Table 2. Detection on the vulnerable/fixed pair for each CVE: whether the tool's output distinguishes the two revisions for the CVE's own mechanism. ✓ = distinguishes vulnerable from fixed; ✗ = does not. The Part 1 pattern baseline is not Tsck; see Section 5.2.

Tool	CVE-2022-4450	CVE-2023-0215	CVE-2024-4741
Part 1 pattern baseline	✓	✓	✓
Clang Static Analyzer 18	✗	✗	✗
Coccinelle 1.1.1 (stock rules)	✗	✗	✗
Cppcheck 2.13.0	✗	✗	✗

Observation 1. Tsck is not limited to the function it was derived from: the unrelated, non-CVE `d2217c88df` KDF fix removes exactly the four violations its own commit changes (`pbkdf1`, `pbkdf2`, `pkcs12kdf`, `scrypt`, each `*buffer`) and adds none, with the rule unchanged from the CVE-2022-4450 derivation.

Observation 2. On current master, Tsck flags 10 sites; manual triage against each site's callers found 2 potential recurrences whose failure branch is not reachable in normal builds, 8 convention deviations with no caller impact, and 0 confirmed new vulnerabilities. (Two of the eight were initially labelled "unclear" pending a caller read; Section 7 reports that re-check.) We report this plainly: the contribution here is a bounded rule that reproduces two real fixes without retuning, not a new bug.

5.6 RQ2: Do general-purpose tools already catch these bugs?

Table 2 compares four tools on six pinned revisions (three CVEs, vulnerable and fixed side). A Clang positive control, run on a synthetic use-after-free, produced the expected diagnostic with the same configuration, confirming the analyzer was active for the clean runs.

Observation 3. None of three general-purpose open-source tools, run with their stock rules and no CVE-specific tuning, distinguishes the vulnerable revision from the fixed revision for any of the three mechanisms. Coccinelle's generic release-event rule fires near `BIO_new_NDEF` in both revisions, but does not inspect `BIO_pop` or the linked-BIO state, so it does not change between versions; Cppcheck's output on the three files is unrelated style diagnostics, identical in count before and after each fix; Clang emits no diagnostic in any of the six runs.

This does not show Tsck beats these tools: Protocol B (Tsck itself) does not implement a rule for two of these three mechanisms (Section 5.1). It shows that a source check specific to one ownership or reset convention is needed at all, because the general-purpose tools' stock configuration does not substitute for it.

5.7 RQ3: Does the rule generalize to an unseen later fix?

The tree at `d2217c88df` still flags `pvk_set_membuf` in `pvkkdf.c`. OpenSSL fixed it one day later, in `a632bfaa4e`, a direct child of `d2217c88df`, with the same `*buffer = NULL` change as the other four KDF sites.

Observation 4. Tsck, run unchanged from its CVE-2022-4450 derivation, flags a site that a different, later commit by OpenSSL went on to fix. This is one site, not a rate, and we report it as a single encouraging case, not as a time-travel precision or recall figure: a systematic time-travel evaluation over more sites is future work.

Table 3. Manual triage of all 11 Protocol A whole-tree flags on the same master snapshot as Table 1. Every site is a false positive; the breakdown is by *why*, not by severity. Full source excerpts and reasoning per site are in `results/protocol_a_triage.md` (Section 9).

Reason	Sites
Confirmed or plausible recurrence	0
Tool limitation (flag-guarded `BIO_pop` the path search cannot correlate)	1
No externally-owned BIO spliced in (freshly-allocated chain only)	10
Total flagged	11

5.8 RQ4: What does Tscк cost in time?

We reran `run_scan.sh` end to end under `/usr/bin/time -v` on a clean checkout, on a different machine from the one that produced Table 1 (Section 5.4). The rerun reproduced every conforming/violation/suppressed count in Table 1 exactly, on the same pinned snapshots. Total wall-clock time was 5 minutes 31 seconds, of which the process held the CPU only 14% of the time (27.17 s user, 20.34 s system); the rest is one-time network latency fetching OpenSSL over a partial clone, not Tscк's own work, and a cached checkout would not pay it again.

Observation 5. Across six snapshots (five full-tree scans of 999–1,155 files each, plus the two single-function pre/post-fix checks), Tscк's own CPU time was under 48 seconds. An independent rerun also reproduced every whole-tree count in Table 1 exactly. We do not break this further into a per-file or per-site rate: the log's CPU figure still mixes Tscк's parsing and search with `git`'s sparse-checkout cost for each snapshot, and separating them would need instrumentation we did not add.

5.9 RQ5: Does a second, differently-shaped rule generalize the same way?

Protocol A checks a different invariant shape than Protocol B: establish-then-required-detach-before-release on a local variable rather than release-then-required-reset on a parameter, with a syntactic alias rule for the common `x = BIO_push(v, . . . )` idiom (Section 5.1). It reuses Protocol B's `tree-sitter` parser and CFG; only the event rule and path search are new. We ran it two ways, both independently reproduced on a separate machine (Section 5.4).

Observation 6. On `BIO_new_NDEF`, Protocol A flags `asn_bio` as a violation before the CVE-2023-0215 fix and as conforming after it, the same pre/post-fix distinction Protocol B makes for CVE-2022-4450. The rule-as-data design extends to a second invariant shape without changing the parser or the CFG builder.

On the same 1,155-file master snapshot as Table 1, Protocol A flags 11 sites. We manually checked every one against the real source at the pinned commit, not a sample.

Observation 7. 0 of the 11 whole-tree violations are confirmed or plausible recurrences. One is a tool limitation: a real `BIO_pop` call exists but is guarded by a boolean flag the path search cannot correlate with the earlier branch that set it. The other 10 release a BIO chain built entirely from freshly-allocated local objects (two of them behind a push call whose failure branch cannot be reached once its first argument is already proven non-`NULL`). None ever splices in an externally-owned BIO before the release – which is what makes CVE-2023-0215's shape dangerous in the first place. Protocol A's rule has no notion of which released object is externally owned, so it cannot tell these apart from the real danger shape. Full per-site source excerpts and reasoning are in the replication package (`results/protocol_a_triage.md`, Section 9).

This is a real result, not a shortfall we are hiding: it confirms, rather than merely predicts, the gap named in Section 8. A single-object rule with no ownership or escape tracking cannot generalize a BIO-chain invariant across a codebase, even though the same architecture generalized a release/reset invariant cleanly in **RQ1**.

6 Discussion

Did Tscк find a new bug? No, and we now have a precise reason why, not just an impression. Two sites on current master have the same shape as CVE-2022-4450 (`X509_PUBKEY_set`, `ossl_ml_kem_i2d_pubkey`; Observation 2). Re-checking both against the real pinned source (`results/reachability_rq1_candidates.md`) found that our original reachability note for `X509_PUBKEY_set` was itself wrong: it blamed a lock/atomic-op failure in `EVP_PKEY_up_ref`, but every platform variant of `CRYPTO_UP_REF` in this codebase always returns `true` – that failure mode is dead code. The real trigger for both sites is the same in shape: the tracked object's state (a reference count, a key's public-key field) must already have changed between two reads that a single straight-line call provably treats as equal. Only a second thread racing the same object, unsynchronized, can cause that change. We checked every in-tree caller of both functions and found none that creates this precondition internally; it requires a pre-existing bug or data race one level up the call stack. We did not build a multithreaded fault-injection test to force that race window, so we still report both as candidates, not findings, but on a corrected and sharper basis than before.

Is this just matching the one patch it was built from? The rule (Section 4.1) names no OpenSSL function. A second, unrelated fix, `d2217c88df`, is detected without changing the rule (Observation 1), and a sibling site that fix missed is flagged and later fixed independently (Observation 4). This is evidence the rule is more than a rewrite of one patch, but it is two data points, not a rate (Section 7). Separately, is the release-without-reset *convention* itself a recurring problem in this codebase, independent of our tool? A commit-message search of `openssl/openssl`'s real history (`results/historical_recurrence_search.md`) turned up ten further, independently-authored fixes of this exact shape. They span nine files in `crypto/`, `providers/`, and `ssl/`, from 2023 to six weeks before our pinned snapshot. None is a parameter or BIO-chain shape either engine currently tracks (mostly struct-field and bare local-variable resets), so this corroborates the convention's generality without being a tool finding.

Why build a custom engine instead of using an existing framework such as CodeQL or Semgrep directly? Table 2 is exactly that attempt, run with each tool's stock configuration. None of three general-purpose open-source tools distinguishes the vulnerable revision from the fixed one for any of the three CVEs we checked (Observation 3); their rule and query sets simply do not encode this

project's specific conventions. We instead write the rule once, as data, over our own control-flow graph (Section 4.1), so we can inspect exactly how it was evaluated.

What about false positives? In the manual triage behind Observation 2, 1 of 18 flagged sites across all six snapshots is a confirmed false positive: `load_common`, which releases only a reference-counted handle's structural reference, not the underlying object (Section 7). Two further sites are suppressed by the tool itself, correctly, under the deref-after-release rule.

Is a clean run on CVE-2023-0215 or CVE-2024-4741 evidence that Tsck handles them? Mixed. Protocol B implements neither mechanism (Section 5.1); the vulnerable/fixed distinction for those two CVEs in Table 2 comes from the Part 1 pattern baseline, not Tsck. Protocol A (**RQ5**, Section 5.9) implements CVE-2023-0215's BIO-chain invariant and distinguishes its own motivating pair. But its whole-tree run finds 0 confirmed recurrences of 11 flags, all false positives from missing ownership tracking – so a clean run elsewhere would not be strong evidence either, for the same reason. CVE-2024-4741 needs a state-guard rule neither shape expresses; future work.

Is the whole-tree conformance rate (91.7% on master) a precision number? No. It is the fraction of release sites that are conforming under one specific, name-based release heuristic (Section 5.5); a site this heuristic cannot see, such as a release behind a struct member, is absent from both the numerator and the denominator. We leave measuring that blind spot for future work.

7 Threats to Validity

Internal threats. These threaten whether Tsck's own results are correct.

- *Tuning on seen data.* The deref-after-release suppression and the `NULL`-guard pruning (Section 4.3) were both added after reading real output on OpenSSL. The second was added specifically after it produced one infeasible-path false positive on the master scan. The master-snapshot numbers in Table 1 are therefore not a blind test of those two refinements, though the rule itself (Section 4.1) was not changed after seeing the KDF result in Observation 1.
- *Release detection is name-based.* A release site is any call matching `(?i)free`. This cannot see reference counts: one site in an earlier snapshot, `load_common`, frees only a structural `ENGINE` reference and is a confirmed false positive under manual review.
- *Parser coverage.* About 39% of scanned files have a `tree-sitter` parse error somewhere, almost always from an unexpanded macro. Only 7 to 8 release sites per snapshot sit inside a function with a parse error, and no flagged violation in Section 5 is among them. A site inside an unparseable function is invisible to Tsck, though, and would not appear in any count we report.
- *Our own triage.* The labels behind Observation 2 (2 potential recurrence, 8 convention deviation) were assigned by the people who built Tsck. We had one independent party re-derive the reachability argument for the 2 potential-recurrence sites from scratch against real source. (The party was a generative AI assistant, not involved in building the rule or running the original scan; see `results/reachability_rq1_candidates.md`.) It found and corrected a factual error in our original note (Section 7 construct threats, below). The same party also re-checked the 2 sites we had labelled "unclear" pending a caller read (`results/unclear_sites_recheck.md`). Both reclassify as convention deviation once the one real in-tree caller (`apps/cmp.c`) is read, since it discards the dangling output on every failure path rather than acting on it; the failure branch itself needs the same unreachable-without-a-race precondition as the two potential-recurrence sites. Every one of the 10 whole-tree flags now has an independently re-derived reason, not just our own. That one

re-check already changed a conclusion, which is itself evidence for, not against, this threat: a human second labeller without knowledge of the rule's derivation remains undone and could still disagree with any of these 10 labels.

What remains: a human independent second labeller on a sample of the triaged sites, and a parser with macro expansion, are both future work.

External threats. These threaten whether the results generalize past our six snapshots and three CVEs.

- *One rule family generalizes within itself, not across invariant shapes.* Protocol B, derived from CVE-2022-4450 (Observation 1, Observation 4), also reproduces an unrelated fix and flags a site a later commit fixed. That is two data points, though, not a sample large enough for a detection rate. Protocol A, a differently-shaped rule for CVE-2023-0215 (Section 5.9), distinguishes its own motivating fix, but its whole-tree run (Observation 7) is 0 for 11. Every flagged site is a false positive, because the rule has no notion of which released object is externally owned. That is a real negative result, not a bug to patch away: a single-object release/detach rule does not, by itself, generalize the way Protocol B's did.
- *Non-independent snapshots.* The six snapshots in Table 1 share most of their long-lived functions across revisions, so the effective sample is closer to 20 distinct release sites than to 20×6.
- *One project.* All subjects (Section 5.1) are OpenSSL. We make no claim about another C codebase with different allocation and ownership conventions.
- *Scope of what is tracked.* Tscк only examines parameters of pointer-to-pointer type or deeper. An out-parameter hidden behind a struct member, such as `ctx->buf`, is invisible to it, however it is released or reset.

What remains: more historical pairs, a second project, tracking struct-member out-parameters, and an ownership/escape analysis that could make Protocol A's whole-tree run precise, are future work.

Construct threats. These threaten whether what we measure is what we claim.

- *A violation is a source-pattern property, not a proven vulnerability.* Tscк reports that some path reaches function exit with `*P` still released; it does not establish that the path is reachable at runtime. Of the 2 potential-recurrence sites in Observation 2, both need the same precondition once independently re-derived against real source (`results/reachability_rq1_candidates.md`): the tracked object's state must change between two reads a single-threaded call proves equal, which only an unsynchronized second thread racing the same object can cause. No in-tree caller creates this precondition. We also found and corrected one factual error in our own earlier note (a claimed lock/atomic-op failure that is dead code on every platform this codebase builds for). We report both as candidates needing a multithreaded fault-injection test, not as bugs.
- *A clean run on a general tool means absence of a diagnostic, not absence of a bug.* Table 2's ✗ cells mean the tool emitted nothing for that translation unit under that configuration (Section 5.2). Clang Static Analyzer, in particular, analyzed only the one affected translation unit per revision, not the whole build. A clean run is therefore a property of that unit and configuration, not a property of the tool in general.
- *RQ2's baseline is not Tscк.* Table 2 compares Clang, Coccinelle, and Cppcheck against the Part 1 pattern matcher, not against Protocol B. A reader who conflates the two would credit Protocol B with a result it did not produce; we repeat this distinction at the point it matters (Section 5.6) to avoid that.

8 Related Work

Rule-guided recurring vulnerability detection. Closest to us is the work of Cao et al. [Cao et al. 2025a], VulPA, which also derives a checkable rule from a known bug and searches a codebase for other violations. Their rule language, VPDL, compiles to an automaton and can bind several correlated variables at once, for example checking that the same object reached both a permission check and a sensitive call; an IFDS-based interprocedural analysis then walks the whole program. VulPA targets Java and reports 90 confirmed new findings across 26 applications and 34 source CVEs. Our Protocol B rule tracks one output parameter's release/reset state, is intraprocedural with no alias analysis, and is evaluated on one rule family and six OpenSSL snapshots, several orders of magnitude smaller. We share their goal of a rule with no hard-coded function names, but our Protocol A, the BIO-chain invariant for CVE-2023-0215 (Section 5.9), is direct evidence for this gap rather than a prediction of it: its whole-tree run is 0 for 11 confirmed recurrences precisely because it lacks the multi-object, ownership-aware correlation VulPA already has and we do not.

Clone-based recurring vulnerability detection. Cao et al. [Cao et al. 2025b] study how well existing recurring-vulnerability detectors (VUDDY, MVP, MOVERY, V1scan, FIRE) find code *reused* from a known-vulnerable function. They show these detectors do much worse on syntactically or semantically modified clones than on exact or renamed ones. Their notion of *recurring* is a vulnerable function's code reappearing, by reuse, elsewhere; ours is a different function independently repeating the same ownership or reset mistake, with no code-clone relationship at all between, for example, `PEM_read_bio_ex` and the four KDF functions in Observation 1. Tan et al. [Tan et al. 2025] address a failure mode specific to clone-based detectors: code that is similar to a vulnerable function but has itself already been patched, and which clone-based tools then misflag. Because Tscк checks a lifecycle rule against control flow rather than matching code similarity, it is not exposed to this particular false-positive source, though Section 7 reports others it is exposed to instead.

Locating when a vulnerability was introduced. Guo and He [Guo and He 2026] identify the commit that first introduced a known vulnerability, by extracting the vulnerable/patched diff's statement-level patching pattern and matching it backward through commit history. This answers a different question from ours: which commit introduced a known bug, not which other functions share its pattern now. We use a single pinned vulnerable/fixed pair per CVE (Section 5.1) rather than search history for one, and take the fix commit as already identified.

Patch porting. Wu et al. [Wu et al. 2025] port an existing, known patch to other versions or branches of the same project, combining semantic and syntactic matching with an LLM. This assumes the target location is already known to need the same fix; our problem is the opposite, finding target locations whose need for the same kind of fix is not yet known. Tscк and Mystique could be complementary: a site Tscк flags as a violation is a candidate location Mystique's porting step could then patch, though we have not built or evaluated that combination.

9 Conclusion

A vulnerability fix is evidence of a convention a project expects its own code to follow, not just a patch to one function. CVE-2022-4450 shows this concretely: `PEM_read_bio_ex` frees two output buffers on a failure path and leaves the caller-visible pointers dangling, and the same release-without-reset shape recurs, unrelated to that fix, in four KDF functions a year earlier. General-purpose static analyzers, run with their stock rule sets, catch none of the three historical CVEs we checked.

We presented Tscк, which reads a lifecycle rule from a data file and checks it against a real `tree-sitter` control-flow graph, rather than matching one patch's text. Protocol B, a release/reset

rule for CVE-2022-4450, flags `PEM_read_bio_ex`'s two outputs before the fix and clears both after it. It also reproduces the unrelated KDF fix without the rule being shown it, and flags a sibling site OpenSSL went on to fix independently. On OpenSSL's current 1,155-file tree it flags 10 sites, of which manual triage confirms 0 new vulnerabilities, 2 potential recurrences needing reachability review, and 8 harmless deviations. Protocol A, a differently-shaped rule for the BIO-chain invariant behind CVE-2023-0215, reuses the same parser and CFG builder and also distinguishes its own motivating fix, but its whole-tree run is 0 for 11: every flagged site is a false positive, for want of ownership tracking the rule's shape cannot express. Together the two rules show Tsck's rule-as-data architecture generalizing across invariant shapes even where a given rule's detection does not generalize across a codebase. We make no claim beyond this: two rules, one project, with the threats in Section 7. An ownership/escape analysis for Protocol A, and a broader historical evaluation, are future work.

Data Availability

This is a named submission (confirmed with the course instructor, 2026-11-13), so the replication package below is the real, public repository, not an anonymized mirror: https://github.com/PASS-IITG/cs5187-term-project-dauntlesscrux.

The replication package contains Tsck's source (`src/tsck/{cfg.py, outparam.py, compare.py}`) and its 19 engine unit tests; the rule as data for Protocol B (`specs/outparam_reset.json`) and Protocol A (`specs/bio_chain_pop.json`, `protocol_a.py`, with its own regression tests in `test_protocol_a_logic.py`); the pinned OpenSSL commit SHAs for every snapshot in Table 1 (`run_scan.sh`, `run_protocol_a.sh`); and the unedited logs, JSON results, and an `environment.txt` recording tool versions behind this paper's numbers (`src/tsck/results/`), including an independent timing rerun's log.

It also contains the manual triage labels and evidence behind every claim in Section 5: `triage.csv`/`triage.md` for Protocol B; `protocol_a_triage.md` for Protocol A, with source excerpts for all 11 whole-tree sites; `reachability_rq1_candidates.md` for the corrected reachability derivation behind the 2 potential-recurrence sites; `unclear_sites_recheck.md` for the caller read that reclassifies the 2 formerly-unclear `OSSL_CMP_get1_rootCaKeyUpdate` sites as convention deviation; and `historical_recurrence_search.md` for 10 independently-authored OpenSSL fixes of the same release-without-reset shape found by a commit-history search. Finally, it includes the tool-comparison evidence behind Table 2. `make check` reruns the offline checks; `make scan` reruns every OpenSSL scan from a clean checkout.

Author Contributions

[TODO: This is a template, not a statement of fact – neither author has filled it in yet. Replace each bracketed line with what that person actually did, in enough detail that it would hold up if either of you were asked about the other's part in the final presentation (per the course rubric, "that was X's part" is not an acceptable answer). This needs to be true, specific, and agreed by both of you before submission, not generated by an assistant.] Rutvij Wagh (230101113): [fill in]. Udayraj Choudhary (230101107): [fill in]. Both authors reviewed and take responsibility for the full paper and replication package, including the sections produced with generative-AI assistance disclosed below.

Use of Generative AI

[TODO: Confirm and finalize before submission. This draft reflects what one assistant was asked to do in one work session. Check it against what both authors actually used across the whole project. Only then treat it as a true statement of record.]

A generative AI assistant ([TODO: name and version]) was used to draft prose in several sections from material the authors had already produced and verified: experiment logs, triage spreadsheets, and tool output under `src/tsck/results/` and `research/`. It also searched for and verified the five related-work citations in Section 8 against publisher pages and DOIs. For Protocol B, it did not run OpenSSL builds, design the rule, or produce any underlying experimental data: all Protocol B numbers were produced by the authors running Tsck and the comparison scripts.

Protocol A (Section 5.9) is an exception needing closer verification. The assistant designed its rule and `specs/bio_chain_pop.json` from the authors' existing CVE-2023-0215 recurrence notes; ran it directly against a real, independently-cloned copy of the pinned OpenSSL source in its own sandboxed environment (`results/protocol_a_run_environment.txt` discloses a slightly different `tree-sitter` version than the project's prior pin); and wrote the false-positive classification for all 11 whole-tree findings in `results/protocol_a_triage.md` after reading the real source at the pinned commit. Authors reproduced that run on a separate machine (Section 5.4), but it was the assistant who performed the first read of the flagged sites and wrote the reasoning. The same caveat applies to three further pieces of primary analysis the assistant produced and wrote up itself, not from author-supplied material. The first is the corrected reachability argument for the 2 potential-recurrence sites in `results/reachability_rq1_candidates.md`, which also corrects a factual error in the authors' own original triage note. The second is the reclassification of the 2 previously-unclear `OSSL_CMP_get1_rootCaKeyUpdate` sites in `results/unclear_sites_recheck.md`, performed by reading the function's one real in-tree caller, `apps/cmp.c`, for the first time. The third is the commit-history search behind `results/historical_recurrence_search.md`. [TODO: Authors must independently re-check the Reason A/B/C classification for each of the 11 Protocol A sites, the reachability argument for all 4 potential-recurrence/formerly-unclear sites, and a sample of the ten historical commits cited, each against its own source before submission, and confirm here, as fact not assumption, that they did – in all four cases the assistant produced the primary analysis, not just prose describing already-verified work.]

References

Liqing Cao, Haofeng Li, Chenghang Shi, Jie Lu, Haining Meng, Lian Li, and Jingling Xue. 2025a. VulPA: Detecting Semantically Recurring Vulnerabilities with Multi-Object Typestate Analysis. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. Accessed: 2026-11-09. doi:10.1145/3729378

Yiheng Cao, Susheng Wu, Ruisi Wang, Bihuan Chen, Yiheng Huang, Chenhao Lu, Zhuotong Zhou, and Xin Peng. 2025b. Recurring Vulnerability Detection: How Far Are We?. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. Accessed: 2026-11-09. doi:10.1145/3728901

Qixuan Guo and Yongzhong He. 2026. Accurate Identification of the Vulnerability-Introducing Commit Based on Differential Analysis of Patching Patterns. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Accessed: 2026-11-09; no DOI shown on the NDSS page at the time of access. https://www.ndss-symposium.org/ndss-paper/accurate-identification-of-the-vulnerability-introducing-commit-based-on-differential-analysis-of-patching-patterns/

Zixuan Tan, Jiayuan Zhou, Xing Hu, Shengyi Pan, Kui Liu, and Xin Xia. 2025. Similar but Patched Code Considered Harmful: The Impact of Similar but Patched Code on Recurring Vulnerability Detection and How to Remove Them. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. Accessed: 2026-11-09; preprint at the given URL, no DOI shown on the ICSE program page at the time of access. https://arxiv.org/abs/2412.20740

Susheng Wu, Ruisi Wang, Yiheng Cao, Bihuan Chen, Zhuotong Zhou, Yiheng Huang, JunPeng Zhao, and Xin Peng. 2025. Mystique: Automated Vulnerability Patch Porting with Semantic and Syntactic-Enhanced LLM. *Proceedings of the ACM on Software Engineering* 2, FSE, Article FSE007 (2025). Accessed: 2026-11-09. doi:10.1145/3715718